

CSC 3201 Module 3 Assignment - Public Key Cryptography Implementation

Marc Helwee and Jaxon Bruzgul

Task 1: Diffie-Hellman Key Exchange

What We Implemented

We implemented Diffie-Hellman key exchange with two parameter sets:

1. Tiny parameters: $q = 37$ and $\alpha = 5$.
2. The supplied 1024-bit IETF q and α values.

Alice and Bob each generate private exponents, calculate public values, and calculate the shared secret:

- Alice: $Y_A = \alpha^{X_A} \bmod q$
- Bob: $Y_B = \alpha^{X_B} \bmod q$
- Shared secret: $s = Y_B^{X_A} \bmod q = Y_A^{X_B} \bmod q$

The shared secret is converted to a 16-byte AES key by calculating SHA-256 of the decimal string representation of s and taking the first 16 bytes. Alice encrypts "Hi Bob!" and Bob encrypts "Hi Alice!" using AES-CBC. Both sides successfully decrypt the other message.

Test Observations

The programs generate new private values each run, so the AES keys and ciphertext values change. The important verification output was:

```
--- Diffie-Hellman (toy params q=37, g=5) ---
```

```
Alice's shared secret == Bob's: True
```

```
Bob  decrypts c0 -> 'Hi Bob!'
```

```
Alice decrypts c1 -> 'Hi Alice!'
```

```
--- Diffie-Hellman (1024-bit IETF group) ---
```

```
Alice's shared secret == Bob's: True
```

```
Bob  decrypts c0 -> 'Hi Bob!'
```

```
Alice decrypts c1 -> 'Hi Alice!'
```

Question 1: How difficult is the Diffie-Hellman problem with the given parameters? What strategy would you use?

With $q = 37$, the discrete logarithm problem is very easy. An attacker can try every possible exponent from 1 through 36 until $\alpha^x \bmod q$ matches the public value. This is a practical brute-force attack.

For the 1024-bit IETF group, a brute-force search is not practical. A generic discrete-log attack such as baby-step giant-step or Pollard's rho would require roughly the square root of the group size. For large prime-field groups, specialized index-calculus or Number Field Sieve techniques are more effective than generic brute force, but they are far beyond a normal classroom machine.

Question 2: Would the same strategy work for both parameter sets?

No. Exhaustive search works immediately for $q = 37$ because the number of possible private exponents is tiny. The same approach is infeasible for a 1024-bit modulus. The large group would require a much more advanced discrete-logarithm attack and substantial computational resources.

Task 2: Diffie-Hellman Man-in-the-Middle Attacks

What We Implemented

Part 1 demonstrates the key-fixing attack. Mallory intercepts Alice and Bob's public values and replaces both values with q . Since $q \bmod q = 0$, both Alice and Bob calculate:

$$s = q^x \bmod q = 0$$

Mallory knows that the shared secret is 0, derives the same AES key, and decrypts both messages.

Part 2 demonstrates generator tampering. Mallory changes α to one of three weak values:

- $\alpha = 1$: Every public value and shared secret is 1.
- $\alpha = q$: Every public value and shared secret is 0.
- $\alpha = q - 1$: The shared secret is either 1 or $q - 1$, depending on the private exponents.

Mallory tries the small set of possible secrets and decrypts both messages.

Test Observations

Part 1: Mallory replaces Y_A and Y_B with q (key fixing)

s computed by Alice : 0

s computed by Bob : 0

s guessed by Mallory : 0

Mallory decrypts c_0 -> 'Hi Bob!'

Mallory decrypts c_1 -> 'Hi Alice!'

Part 2: Mallory tampers with the generator g

g set to 1: s_alice==s_bob True, candidates [1]

Mallory decrypts c0 with s=1: 'Hi Bob!'

Mallory decrypts c1 with s=1: 'Hi Alice!'

g set to q: s_alice==s_bob True, candidates [0]

Mallory decrypts c0 with s=0: 'Hi Bob!'

Mallory decrypts c1 with s=0: 'Hi Alice!'

g set to q-1: s_alice==s_bob True, candidates [1, q-1]

Mallory decrypts c0 with s=1: 'Hi Bob!'

Mallory decrypts c1 with s=1: 'Hi Alice!'

For $\alpha = q - 1$, the successful candidate can be either 1 or $q - 1$ on different runs. The code tests both possible values.

Question 3: Why are the attacks possible, and how can they be prevented?

The attacks work because unauthenticated Diffie-Hellman does not prove who sent the public values or parameters. Mallory can modify the exchanged public values or replace the generator before Alice and Bob calculate their shared secret.

The attacks can be prevented by authenticating the Diffie-Hellman exchange. For example, Alice and Bob can sign the exchanged public values and transcript using certificates, or authenticate the exchange with a pre-shared-key MAC. Applications should use trusted, fixed group parameters and validate received public values before using them.

Task 3: RSA Encryption, Malleability, and Signatures

What We Implemented:

We implemented textbook RSA key generation using $e = 65537$.

The program:

- Generates two distinct random primes p and q .
- Computes $n = p \times q$.
- Computes $\phi(n) = (p - 1)(q - 1)$.
- Uses an extended Euclidean algorithm to calculate $d = e^{-1} \bmod \phi(n)$.
- Encrypts with $c = m^e \bmod n$.
- Decrypts with $m = c^d \bmod n$.

The program uses 1024-bit primes for a 2048-bit RSA modulus. The key-generation function accepts prime sizes up to 2048 bits.

We also demonstrate an RSA key-exchange attack. Bob encrypts a secret, but Mallory replaces the ciphertext with $\text{Encrypt}(r)$, where r is a value Mallory selected. Alice decrypts the

replacement ciphertext and derives an AES key from r . Mallory knows r , derives the same AES key, and decrypts Alice's message.

A second RSA integrity example shows that raw RSA encryption is multiplicative. If Mallory intercepts $\text{Encrypt}(1000)$, she can multiply it by $2^e \bmod n$. Alice decrypts the altered ciphertext as 2000.

Finally, we demonstrate raw RSA signature forgery. Given signatures for m_1 and m_2 , Mallory multiplies the signatures to create a valid signature for:

$$m_3 = m_1 \times m_2 \bmod n$$

Test Observations

Part 1: textbook RSA key gen + encrypt/decrypt

prime size: 1024 bits each, modulus n : 2048 bits, $e = 65537$

'Hi Bob!' -> ... -> 'Hi Bob!' ok=True

'RSA works' -> ... -> 'RSA works' ok=True

'malleable?' -> ... -> 'malleable?' ok=True

Part 2: Mallory fixes the RSA key exchange via malleability

Mallory chose $r = 12345$, replaced c with $c' = \text{Encrypt}(r)$

Alice's $s' = c'^d \bmod n = 12345$ ($= r$: True)

keys match: True

Mallory decrypts c_0 -> 'Hi Bob!'

Tamper: $\text{enc}(1000) * 2^e$ decrypts to 2000 ($= 2 * 1000$: True)

-> an attacker can silently alter a value

Part 3: RSA signature forgery via malleability

captured $\text{Sign}(42)$, $\text{Sign}(1337)$

$$m_3 = 42 * 1337 = 56154$$

$$\text{forged Sign}(42 * 1337) = \text{sig}_1 * \text{sig}_2 \bmod n$$

verifies as valid: True

Question 4: Why is it bad practice for multiple users to share the same RSA modulus n , even if they use the same common public exponent e ?

Each RSA user should have a separate modulus. If multiple users share n and one private key or factor of n is exposed, the shared modulus is compromised for everyone using it. Knowledge of a valid private exponent can also be used to factor the shared modulus.

Sharing a modulus can also enable common-modulus attacks when the same plaintext is encrypted using the same modulus with different public exponents. Using the same public exponent does not make sharing n safe; it only removes that one specific form of the common-modulus attack. Separate moduli isolate each user's key material and limit the effect of a compromise.

CODE:

task1_dh.py

```
import hashlib
import secrets

from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

IV = bytes(16)

def derive_key(s):
    return hashlib.sha256(str(s).encode()).digest()[:16]

def aes_encrypt(key, message):
    cipher = AES.new(key, AES.MODE_CBC, IV)
    return cipher.encrypt(pad(message.encode(), AES.block_size))

def aes_decrypt(key, ciphertext):
    cipher = AES.new(key, AES.MODE_CBC, IV)
    return unpad(cipher.decrypt(ciphertext), AES.block_size).decode()

def diffie_hellman(q, g, label=""):
    print(f"--- Diffie-Hellman {label} ---")
    g_display = str(g) if g.bit_length() <= 64 else f"{g.bit_length()}-bit value"
    print(f" public params: q ({q.bit_length()} bits), g = {g_display}")

    xa = secrets.randbelow(q - 2) + 1
    xb = secrets.randbelow(q - 2) + 1

    ya = pow(g, xa, q)
    yb = pow(g, xb, q)
    s_alice = pow(yb, xa, q)
    s_bob = pow(ya, xb, q)

    print(f" Alice's shared secret == Bob's: {s_alice == s_bob}")
    ka, kb = derive_key(s_alice), derive_key(s_bob)
    print(f" Alice's AES key: {ka.hex()}")
    print(f" Bob's AES key: {kb.hex()}")
```

```

assert ka == kb, "key agreement failed"

c0 = aes_encrypt(ka, "Hi Bob!")
c1 = aes_encrypt(kb, "Hi Alice!")
print(f" Bob  decrypts c0 -> {aes_decrypt(kb, c0)!r}")
print(f" Alice decrypts c1 -> {aes_decrypt(ka, c1)!r}")
print()
return ka

IETF_Q = int(
    "B10B8F96A080E01DDE92DE5EAE5D54EC52C99FBCFB06A3C69A6A9DCA52D23B61"
    "6073E286"
    "75A23D189838EF1E2EE652C013ECB4AEA906112324975C3CD49B83BFACCBDD7D"
    "90C4BD7098488E9C219A73724EFFD6FAE5644738FAA31A4FF55BCCC0A151AF5F"
    "0DC8B4BD45BF37DF365C1A65E68CFDA76D4DA708DF1FB2BC2E4A4371", 16)

IETF_G = int(
    "A4D1CBD5C3FD34126765A442EFB99905F8104DD258AC507FD6406CFF14266D31"
    "266FEA1E5C41564B777E690F5504F213160217B4B01B886A5E91547F9E2749F4"
    "D7FBD7D3B9A92EE1909D0D2263F80A76A6A24C087A091F531DBF0A0169B6A28A"
    "D662A4D18E73AFA32D779D5918D08BC8858F4DCEF97C2A24855E6EEB22B3B2E5", 16)

if __name__ == "__main__":
    diffie_hellman(37, 5, label="(toy params q=37, g=5)")
    diffie_hellman(IETF_Q, IETF_G, label="(1024-bit IETF group)")

```

task2_mitm.py

```

import secrets

from task1_dh import derive_key, aes_encrypt, aes_decrypt, IETF_Q, IETF_G

def recover_message(ciphertext, candidates, expected_message):
    for s_guess in candidates:
        try:
            recovered = aes_decrypt(derive_key(s_guess), ciphertext)
        except ValueError:
            continue
        if recovered == expected_message:
            return s_guess, recovered
    raise AssertionError("Mallory could not recover the expected message")

def describe_secret(secret, q):
    return "q-1" if secret == q - 1 else str(secret)

def part1_key_fixing(q, g):
    print("Part 1: Mallory replaces Y_A and Y_B with q (key fixing)")

```

```

xa = secrets.randbelow(q - 2) + 1
xb = secrets.randbelow(q - 2) + 1
_ya, _yb = pow(g, xa, q), pow(g, xb, q)

yb_seen_by_alice = q
ya_seen_by_bob = q

s_alice = pow(yb_seen_by_alice, xa, q)
s_bob = pow(ya_seen_by_bob, xb, q)
s_mallory = 0

print(f" s computed by Alice : {s_alice}")
print(f" s computed by Bob   : {s_bob}")
print(f" s guessed by Mallory : {s_mallory}")
assert s_alice == s_bob == s_mallory

c0 = aes_encrypt(derive_key(s_alice), "Hi Bob!")
c1 = aes_encrypt(derive_key(s_bob), "Hi Alice!")
k_m = derive_key(s_mallory)
print(f" Mallory decrypts c0 -> {aes_decrypt(k_m, c0)!r}")
print(f" Mallory decrypts c1 -> {aes_decrypt(k_m, c1)!r}")
print()

```

```

def part2_generator_tampering(q):
    print("Part 2: Mallory tampers with the generator g")

    for g_bad in (1, q, q - 1):
        xa = secrets.randbelow(q - 2) + 1
        xb = secrets.randbelow(q - 2) + 1
        ya, yb = pow(g_bad, xa, q), pow(g_bad, xb, q)
        s_alice = pow(yb, xa, q)
        s_bob = pow(ya, xb, q)

        if g_bad == 1:
            candidates = [1]
        elif g_bad == q:
            candidates = [0]
        else:
            candidates = [1, q - 1]

        assert s_alice == s_bob
        c0 = aes_encrypt(derive_key(s_alice), "Hi Bob!")
        c1 = aes_encrypt(derive_key(s_bob), "Hi Alice!")
        s0, recovered0 = recover_message(c0, candidates, "Hi Bob!")
        s1, recovered1 = recover_message(c1, candidates, "Hi Alice!")

        tag = "q" if g_bad == q else ("q-1" if g_bad == q - 1 else "1")
        candidate_text = ", ".join(describe_secret(candidate, q) for candidate in candidates)
        print(f" g set to {tag:>3}: s_alice==s_bob {s_alice == s_bob}, "
              f"candidates [{candidate_text}]")
        print(f" Mallory decrypts c0 with s={describe_secret(s0, q)}: {recovered0!r}")
        print(f" Mallory decrypts c1 with s={describe_secret(s1, q)}: {recovered1!r}")
    print()

```

```
if __name__ == "__main__":
    part1_key_fixing(IETF_Q, IETF_G)
    part2_generator_tampering(IETF_Q)
```

task3_rsa.py

```
from Crypto.Util.number import getPrime
```

```
from task1_dh import derive_key, aes_encrypt, aes_decrypt
```

```
E = 65537
```

```
def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) * y1
```

```
def modinv(a, m):
    g, x, _ = egcd(a % m, m)
    if g != 1:
        raise ValueError("no modular inverse (a and m not coprime)")
    return x % m
```

```
def generate_keypair(prime_bits=1024):
    if not 2 <= prime_bits <= 2048:
        raise ValueError("prime_bits must be between 2 and 2048")
    while True:
        p, q = getPrime(prime_bits), getPrime(prime_bits)
        if p == q:
            continue
        n = p * q
        phi = (p - 1) * (q - 1)
        if egcd(E, phi)[0] == 1:
            d = modinv(E, phi)
            return (n, E), (n, d)
```

```
def encrypt(m, pub):
    n, e = pub
    assert 0 <= m < n, "message integer must be < n"
    return pow(m, e, n)
```

```
def decrypt(c, priv):
    n, d = priv
    return pow(c, d, n)
```

```

def str_to_int(s):
    return int.from_bytes(s.encode(), "big")

def int_to_str(i):
    return i.to_bytes((i.bit_length() + 7) // 8, "big").decode()

def part1_roundtrip():
    print("Part 1: textbook RSA key gen + encrypt/decrypt")
    pub, priv = generate_keypair(prime_bits=1024)
    n = pub[0]
    print(f" prime size: 1024 bits each, modulus n: {n.bit_length()} bits, e = {E}")
    for msg in ["Hi Bob!", "RSA works", "malleable?"]:
        m = str_to_int(msg)
        c = encrypt(m, pub)
        back = int_to_str(decrypt(c, priv))
        print(f" {msg!r:14} -> c={str(c)[:24]}... -> {back!r} ok={back == msg}")
    print()
    return pub, priv

def part2_malleability_mitm(pub, priv):
    print("Part 2: Mallory fixes the RSA key exchange via malleability")
    n, e = pub

    s_bob = 0xDEADBEEFCAFE
    c_bob = encrypt(s_bob, pub)

    r = 12345
    c_prime = encrypt(r, pub)

    s_alice = decrypt(c_prime, priv)
    k_alice = derive_key(s_alice)
    c0 = aes_encrypt(k_alice, "Hi Bob!")

    k_mallory = derive_key(r)
    assert s_alice == r
    assert k_alice == k_mallory
    print(f" Bob sends c = Encrypt({s_bob}) (starts {str(c_bob)[:24]}...)")
    print(f" Mallory chose r = {r}, replaced c with c' = Encrypt(r)")
    print(f" Alice's s' = c'^d mod n = {s_alice} (== r: {s_alice == r})")
    print(f" keys match: {k_alice == k_mallory}")
    print(f" Mallory decrypts c0 -> {aes_decrypt(k_mallory, c0)!r}")

    m = 1000
    c = encrypt(m, pub)
    c_tampered = (c * pow(2, e, n)) % n
    m_tampered = decrypt(c_tampered, priv)
    print(f" Tamper: enc(1000) * 2^e decrypts to {m_tampered} "
          f" (= 2*1000: {m_tampered == 2000})")
    print(" -> an attacker can silently alter a value")
    print()

```

```

def part3_signature_forgery(pub, priv):
    print("Part 3: RSA signature forgery via malleability")
    n = pub[0]

    def sign(m):
        return decrypt(m, priv)

    def verify(m, sig):
        return encrypt(sig, pub) == m % n

    m1, m2 = 42, 1337
    sig1, sig2 = sign(m1), sign(m2)

    m3 = (m1 * m2) % n
    forged = (sig1 * sig2) % n

    print(f" captured Sign({m1}), Sign({m2})")
    print(f" m3 = {m1} * {m2} = {m3}")
    print(f" forged Sign({m1}*{m2}) = sig1*sig2 mod n")
    print(f" verifies as valid: {verify(m3, forged)}")
    print()

if __name__ == "__main__":
    pub, priv = part1_roundtrip()
    part2_malleability_mitm(pub, priv)
    part3_signature_forgery(pub, priv)

```